



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Sebastian Alexander Villeda Reyna **Carné:** 1032625

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
R// Es un conjunto de reglas y protocolos que permite que diferentes aplicaciones de software se comuniquen entre sí a través de internet de manera sencilla y eficiente
2. ¿Qué es un endpoint?
R// Es una dirección digital específica (una URL) en un servidor donde una aplicación puede conectarse para enviar o recibir datos
3. ¿Qué diferencia existe entre una URL y un endpoint?
R// Una URL es cualquier dirección web general que sirve para ubicar un recurso en internet, a diferencia del endpoint que es una URL específica que esta combinada con un método HTTP, el cual está diseñado exclusivamente para una app o API.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
R//
GET: Pide y recupera datos sin modificar nada.
POST: Se encarga de enviar los datos nuevos al servidor para crear registros.
PUT: Actualiza el dato que se desee, lo elimina por completo y lo reemplaza por el que se quiera.
DELETE: Borra el dato/registro indicado.
5. ¿Qué es JSON y por qué es común en las APIs?
R// Es un formato de texto ligero basado en pares de clave-valor y listas ordenadas que sirve para almacenar y transferir datos estructurados.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
R// Es una respuesta numérica de tres dígitos que el servidor envía al navegador o cliente para indicar el resultado de una solicitud.

El 200 indica que la solicitud del cliente se ha completado con éxito y el servidor devuelve el recurso solicitado. Como datos en forma JSON.

El 404 significa que el servidor no encontró el recurso que se andaba buscando, un ejemplo es a la hora de colocar una URL y que está este mal escrita.

El 500 señala que hay un error genérico en el servidor, ya que indica que el servidor encontró una situación inesperada y no pudo completar la solicitud.
7. ¿Qué información se puede encontrar en la respuesta de una API?
R// Contiene la información que el servidor devuelve tras procesar una solicitud hecha por el cliente.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
R// Tiene como ventajas probar y simular una API antes de escribir el código de programación ofrece grandes ventajas en el desarrollo de software.



9. ¿Qué es y para qué sirve Swagger UI?

R// Es una herramienta de código abierto que genera una interfaz web interactiva y visual a partir de la especificación OpenAPI de una API REST.

Sirve como documentación visual automática, pruebas en tiempo real, facilita la colaboración y ahorro de tiempo.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

R// Funciona como una plataforma integral para diseñar, enviar solicitudes, probar, documentar y gestionar APIs a lo largo de todo su ciclo de vida.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?



- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si



Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.